



Stackelberg Game-Based Hybrid Charging Solution: Simulation Setup and Technical Documentation

Version 1.0

Joy Dutta

February 15, 2025

Contents

1	Introduction	2
2	Initial Setup	2
2.1	Simulation Parameters	2
2.2	EV Characteristics	2
3	Real-World Considerations	3
3.1	Realistic Simulation Setup	3
4	Cost Functions in the Stackelberg Game	3
4.1	EV Cost Function	3
4.2	Leader's Profit Function	3
5	EV Request Handling	4
5.1	Handling Process	4
6	MCV Behavior and Dynamic Deployment	4
6.1	Scenario-based Deployment	4
7	Dynamic Deployment Logic	5
7.1	Triggering Conditions	5
7.2	Decision Flow	5
8	Example Execution	6
8.1	Simulation Initialization	6
8.2	Real-Time Request Handling	6
8.3	Dynamic Deployment Example	6
9	Understand Implementation	6
9.1	High-Level Workflow of the Simulation	6
9.2	Function-by-Function Breakdown	8
9.3	Simulation Start to End: Overview	12
9.4	Simulation at a Glance	13

List of Code Listings

1 Introduction

This document provides a comprehensive overview of the simulation setup, covering the initial configuration, real-world considerations, cost functions in the Stackelberg game, and the dynamic behavior of mobile charging vehicles (MCVs). Special emphasis is placed on EV request handling and adaptive MCV deployment logic.

2 Initial Setup

2.1 Simulation Parameters

- **Simulation Area:** 20 km x 20 km.
- **Simulation Duration:** 8 hours (480 minutes).
- **Fixed Charging Stations (FCS):**
 - Total: 3 stations.
 - Capacity: 10 ports per station.
 - Charging Rate: 100 kWh/hour.
 - Fixed Price: 0.3 currency units per kWh.
- **Mobile Charging Vehicles (MCVs):**
 - Total: 6 available MCVs.
 - Capacity: 4 ports per MCV.
 - Battery Capacity: 900 kWh.
 - Charging Rate: 150 kWh/hour.
 - Deployment Cost: 20 currency units per deployment.
 - Travel Cost: 2 currency units per km.

2.2 EV Characteristics

- **Energy Demand:** Randomly sampled from 30–70 kWh.
- **Location:** Randomly distributed within the simulation area.

- **Waiting Time Constraints:**
 - Maximum at FCS: 90 minutes.
 - Maximum at MCV: 60 minutes.

3 Real-World Considerations

3.1 Realistic Simulation Setup

- **Demand Distribution:** EV arrivals follow a Poisson process, reflecting stochastic real-world demand.
- **Dynamic Pricing for MCVs:** Prices range between 1.1x and 2.0x the FCS fixed price, adjusted based on realistic factors like deployment cost, operational cost, and travel cost.
- **Geographical Constraints:** Road distances include a scaling factor to account for non-linear travel paths.
- **Battery SOC Management:** MCVs return to the depot for recharging when SOC drops below 50 kWh.
- **Traffic and Congestion:** Travel speeds for MCVs are limited to 40 km/h.

4 Cost Functions in the Stackelberg Game

4.1 EV Cost Function

$$C_{EV} = \zeta \cdot C_{\text{charging}} + \delta \cdot C_{\text{travel}} + \epsilon \cdot C_{\text{wait}}, \quad (1)$$

- C_{charging} : Charging cost ($kWh \times \text{price/unit}$).
- C_{travel} : Travel cost ($\text{distance} \times \text{travel cost/unit distance}$).
- C_{wait} : Wait time cost ($\text{wait time (hours)} \times \text{hourly cost}$).

4.2 Leader's Profit Function

$$\text{Profit} = R_{\text{MCVs}} + R_{\text{FCSs}} - C_{\text{deployment}} - C_{\text{travel}}, \quad (2)$$

- R_{MCVs} : Revenue from MCV charging.

- R_{FCSs} : Revenue from FCS charging.
- $C_{\text{deployment}}$: Deployment cost of MCVs.
- C_{travel} : Travel cost of MCVs.

5 EV Request Handling

5.1 Handling Process

1. Case 1: Assign to FCS

- Evaluate travel distance and cost to all FCSs.
- Calculate expected wait time.
- Assign to the FCS with the lowest total cost if wait time ≤ 90 minutes.

2. Case 2: Assign to Existing MCV

- Check available MCVs and their meeting points.
- Evaluate travel distance, cost, and expected wait time.
- Assign to the MCV with the lowest total cost (effective cost) if wait time ≤ 60 minutes.

3. Case 3: Deploy New MCV

- Check demand at meeting points and predicted future demand.
- Deploy a new MCV if at least two unserved EVs justify deployment.

4. Case 4: EV Not Served

- Mark EV as not served if no suitable FCS or MCV is available.

6 MCV Behavior and Dynamic Deployment

6.1 Scenario-based Deployment

1. High Utilization at FCS

- Deploy an MCV to the nearest meeting point when FCS utilization exceeds 100% for 10 minutes.

2. **Low Utilization of MCV**

- Return idle MCVs to the depot after 15 minutes of low utilization with no waiting EVs.

3. **Peak Hours**

- Prioritize staying deployed near high-demand areas.

4. **Off-Peak Hours**

- Redeploy MCVs to regions with consistent demand or return them to the depot.

5. **Battery SOC Below Threshold**

- Return to the depot for recharging.

7 **Dynamic Deployment Logic**

7.1 **Triggering Conditions**

- **FCS 100% Utilization:** Deploy an MCV after sustained full utilization for 10 minutes.
- **Idle MCVs:** Keep idle MCVs near high-demand areas during peak hours.

7.2 **Decision Flow**

1. Monitor real-time utilization.
2. Prioritize deployment based on proximity.
3. Redeploy or return to depot based on utilization thresholds.

8 Example Execution

8.1 Simulation Initialization

- 60 EVs are pre-allocated to FCSs and MCVs, with arrival times set to 0 minutes.
- MCVs are deployed to meeting points based on initial demand.

8.2 Real-Time Request Handling

1. An EV requests 50 kWh at location (10, 15). 2. The system evaluates:

- FCS 0: $C_{EV} = 30 + 10 + 20 = 60$.
- MCV 1 at Meeting Point A: $C_{EV} = 45 + 8 + 15 = 68$.

3. EV is assigned to FCS 0 as it has the lowest cost.

8.3 Dynamic Deployment Example

- FCS 2 reaches 100% utilization for 15 minutes.
- Demand at nearby meeting point exceeds the threshold.
- MCV 4 is deployed to Meeting Point C to relieve congestion.

9 Understand Implementation

In this section, we provide a comprehensive overview of how the entire code is organized and how each function or sub-process interacts with one another. By reading this, one should be able to follow the entire program flow and understand precisely how the EV Charging Simulation is implemented.

9.1 High-Level Workflow of the Simulation

1. Initialization Phase:

- The code sets and loads the following global variables and configurations:
 - `simulation_duration`: total minutes to simulate (e.g., 8 hours = 480 minutes).

- `max_num_mcv`: maximum number of Mobile Charging Vehicles (MCVs) that can be deployed.
- `battery_capacity_mcv`, `minimum_capacity_mcv`, and `capacity_fcs`: battery and capacity constraints.
- `average_ev_demand`, `cost_per_km_ev`, `fixed_charging_price`, and other cost/price parameters.
- `c_mcv` and `c_fcs`: maximum simultaneous charging ports for MCVs and FCSs, respectively.
- `zeta`, `delta`, `epsilon`: weights for cost function components.
- `cost_per_hour`, `charging_rate_mcv`, `charging_rate_fcs`, and maximum wait times.
- `theta`, `theta_max`: constraints that define MCV pricing relative to FCS pricing.
- `cost_per_km_mcv` and `deployment_cost_mcv`.
- Loads or initializes CSV files containing:
 - FCS information (`fcs_locations`, possibly with assigned EVs).
 - Meeting points (`meeting_points` and `meeting_point_prices`).
 - EV data if provided (or empty if not).
 - MCV initial setup, if any (defines existing MCV states).
- Creates and populates dictionaries in `results` to track EV details, waiting times, usage, not-served counts, and more.
- Creates a SimPy environment: `env = simpy.Environment()`.

2. Starting Processes:

- Launches `env.process(ev_arrival_process(env))`, which governs all EV arrivals over time.
- Launches `env.process(mcv_status_monitor(env))`, which periodically monitors MCV status.

- Launches minute-by-minute logging processes `log_fcs_status(env)` and `log_mcv_status(env)` to record utilization.

3. Simulation Run:

- The main run call is `env.run(until=simulation_duration)`.
- During the run:
 - (a) EVs arrive each minute via `ev_arrival_process`.
 - (b) Each EV calls `assign_ev_to_station` to find an FCS or MCV; if no option is feasible, it is marked *not-served*.
 - (c) If assigned, the EV processes:
 - `ev_process_fcs` for FCS charging.
 - `ev_process_mcv` for MCV charging.
 - (d) MCV logic (e.g., battery depletion, underutilization) can trigger `mcv_return_to_depot`.
 - (e) The simulation can dynamically deploy new MCVs (`deploy_new_mcv` or `update_mcv_deployment`), and reassign EVs if newly available MCV capacity appears.

4. End of Simulation:

- Once `simulation_duration` is reached, the environment stops.
- The code then finalizes logs and calculates:
 - Utilization for each FCS/MCV over different time periods (Peak, Normal, Off-Peak).
 - Leader's profit, from MCV and FCS revenue minus deployment/travel costs.
- The code generates summary figures, saves CSV outputs, and prints final usage statistics. Finally, it closes the log file.

9.2 Function-by-Function Breakdown

A. Plotting/Visualization Functions

- `plot_simulation_state(ax, t, draw_lines=False)`: Plots the simulation state at minute `t`. Shows depot, FCS, meeting points, MCVs, EVs, and optionally draws lines from EV to FCS/MCV.
- `plot_current_state(t)`: Similar to `plot_simulation_state` but used when creating static images at specific times.
- `plot_video1_state(t)`, `plot_video2_state(t)`, `plot_video3_state(t)`: Each of these is used for generating frames for different videos (e.g., EV Arrivals, Assignment Mapping, Full Dynamics). They set up an axis, plot relevant data for time `t`, and the code then calls `writerX.grab_frame()`.

B. Simulation Process Functions

- `ev_arrival_process(env)`:
 - Repeats every simulated minute.
 - Determines how many EVs arrive based on a Poisson distribution (rate depends on time of day).
 - For each EV: creates an entry in `results[ ' evs' ]`, updates counters (e.g., `total_evs_requested`, `ev_charge_requested`), and calls `assign_ev_to_station`.
- `assign_ev_to_station(env, ev_id, ev_loc, ev_demand, reassignment=False)`
 - Collects cost-based *options* from `find_fcs_options`, `find_mcv_options`, and `find_new_mcv_options`.
 - Chooses the best option or marks the EV *not-served*.
- `ev_process_fcs(env, ev_id, ev_demand, chosen_fcs, results)`:
 - SimPy process that simulates EV waiting and charging at an FCS.
 - If the wait exceeds `max_wait_time_fcs`, the EV is *not-served*.
 - Otherwise, logs waiting time, charging time, updates results.
- `ev_process_mcv(env, ev_id, ev_demand, chosen_mcv_id, results)`:
 - Similar to `ev_process_fcs` but for MCV:
 - MCV battery (SOC) is decremented by the EV demand.
 - If MCV is underutilized or low on battery, triggers `mcv_return_to_depot`.

- `mcv_return_to_depot(env, mcv_id)`:
 - Handles MCV returning to the depot, traveling time, recharging the MCV, and checking if demand is still high enough for redeployment.
- `mcv_deployment(env, mcv_id)`:
 - Deploys the MCV from depot to a meeting point (travel time included).
 - Activates the MCV (`mcv[ 'available' ] = True`) after arrival.
- `mcv_status_monitor(env)`:
 - Runs every 15 minutes.
 - Logs each MCV's status (location, SOC, number of EVs waiting/charging) in `mcv_status_log`.

C. Logic Helper Functions

- `find_fcs_options(ev_loc, ev_demand), find_mcv_options(ev_loc, ev_demand), find_new_mcv_options(...)`:
 - Return cost and feasibility data for each potential charging route (FCS, existing MCV, or new MCV deployment).
- `deploy_new_mcv(env, ev_id, chosen_option)`:
 - If the system chooses to create a new MCV for certain meeting point and EV, it sets up the MCV state and starts a `delayed_ev_process`.
- `mark_ev_not_served(...)`:
 - Marks an EV as not served and updates logs (time period, unserved counters, etc.).
- `mcv_future_energy_demand(mcv)`:
 - Computes total assigned EV demand on that MCV to ensure the MCV has enough battery left.
- `update_remaining_charging_times(...)`:
 - Updates charging time left for EVs, if needed (not always heavily used).

- `should_mcv_return(mcv, current_time, duration=15, utilization_threshold)`
 - Checks logs from `mcv_minute_status` to see if MCV usage is too low. Suggests returning to depot if it is idle beyond threshold.
- `return_idle_mcv_to_depot(env)`:
 - Looks for MCVs idle for more than 60 minutes with no allocations, triggers `mcv_return_to_depot`.
- `update_mcv_deployment(env)`:
 - Checks for certain conditions (like high FCS queues, 100% FCS utilization for multiple minutes) and decides whether to deploy more MCVs or reassign EVs.
 - Reassigns previously not-served EVs if new MCVs become available.

D. Logging/Data-Collection Functions

- `log_fcs_status(env)`:
 - Runs every minute, logging how many EVs are in queue or charging at each FCS.
 - Stores usage data in `fcs_minute_status`.
- `log_mcv_status(env)`:
 - Similarly logs MCV usage every minute in `mcv_minute_status`.
- `update_mcv_status(t)`:
 - Appends each MCV's location, availability, queue length, etc., to `results['mcv_status_log']`.

E. Post-Simulation Calculation/Reporting

- `calculate_utilization(...)`:
 - Computes busy time vs capacity time for both FCS and MCV in each time period (Peak, Normal, Off-Peak).
- `calculate_leaders_profit(results)`:

- Adds up MCV and FCS revenues, subtracts MCV deployment cost and travel cost, returns the net profit.
- *Saving CSVs and Final Printouts:*
 - The code compiles each EV's data, logs FCS or MCV usage, and writes them to CSV files (e.g., `EV_Data.csv`, `FCS_Data.csv`, `MCV_Data.csv`).
 - Finally, it prints an overall summary (total EV requests, not-served EVs, final profit) and closes the log.

9.3 Simulation Start to End: Overview

- **Global Setup:**
 - The code sets up parameters (charging rates, capacities, etc.), reads or initializes CSV data for FCS, MCV, EV, and other inputs.
 - Creates a SimPy environment and schedules background processes.
- **EV Arrivals:**
 - The process `ev_arrival_process` starts generating EVs one minute at a time.
 - For each EV, the code attempts to find a suitable station (FCS or MCV).
 - If none is found, the EV is marked as *not_served*.
- **Charging:**
 - If assigned to an FCS, the EV executes `ev_process_fcs`, possibly queuing if all FCS ports are busy.
 - If assigned to an MCV, the EV executes `ev_process_mcv`, and the MCV's battery is decremented by the EV's demand.
- **MCV Deployment:**
 - The system can dynamically deploy new MCVs or reassign EVs based on conditions (e.g., a high queue at FCS, or cost advantages of an MCV).

- MCVs can return to the depot if they are underutilized or have low battery.
- **Recording:**
 - Every minute, the code logs usage for FCS and MCV in `fcs_minute_status` and `mcv_minute_status`.
 - Every 15 minutes, `mcv_status_monitor` records a snapshot of MCV status.
- **Stop:**
 - After `simulation_duration` minutes (8 hours), the SimPy environment stops.
- **Post-Processing:**
 - The code calculates utilization, profit, etc., generates figures, saves CSV logs, and prints a summary.
 - All logs are closed, and the script terminates.

In summary, the code is a hybrid discrete-event simulation that uses SimPy to manage concurrency and waiting logic. Its main functionalities include:

- Generating EVs (some may go unserved),
- Assigning EVs to either FCS or MCV,
- Dynamically monitoring and deploying MCV resources,
- Logging all usage statistics,
- Finally producing results and calculating profitability metrics.

9.4 Simulation at a Glance

1. **Initialize Global Parameters and Structures:** The code loads or initializes every parameter and CSV. Then it sets up the SimPy environment.
2. **Start Processes:** The main logic for EV arrivals, MCV monitoring, and per-minute logging kicks off.
3. **Run the Simulation Until `simulation_duration`:**

- EV arrivals (`ev_arrival_process`) generate EVs each minute, calling `assign_ev_to_station`.
- If assigned to FCS, the EV calls `ev_process_fcs`; if assigned to an MCV, it calls `ev_process_mcv`.
- MCVs can redeploy or return to depot if conditions (e.g., battery or utilization) dictate.
- `log_fcs_status` and `log_mcv_status` gather usage data minute by minute.

4. **Stop Simulation and Produce Outputs:**

- After 8 hours (480 minutes), the simulation ends.
- The code calculates final utilization metrics, prints the number of served vs. not-served EVs, and computes the Leader's profit.
- It outputs CSV logs, static figures, and video animations if applicable.
- The program then closes and logs are completed.